INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 2 LECTURE

SYSTEM STARTUP & SHELL PROGRAMMING

WISH YOU ALL THE BEST

SOC 3010
OPERATING SYSTEM

WEEK 2 LECTURE 1

SYSTEM STARTUP

SYSTEM STARTUP

Bootstrapping

Q) **What happens right after users have switched on their computers, that is, how a Linux kernel image is copied into memory and executed. In short, how the kernel, and thus the whole system, is "bootstrapped."**

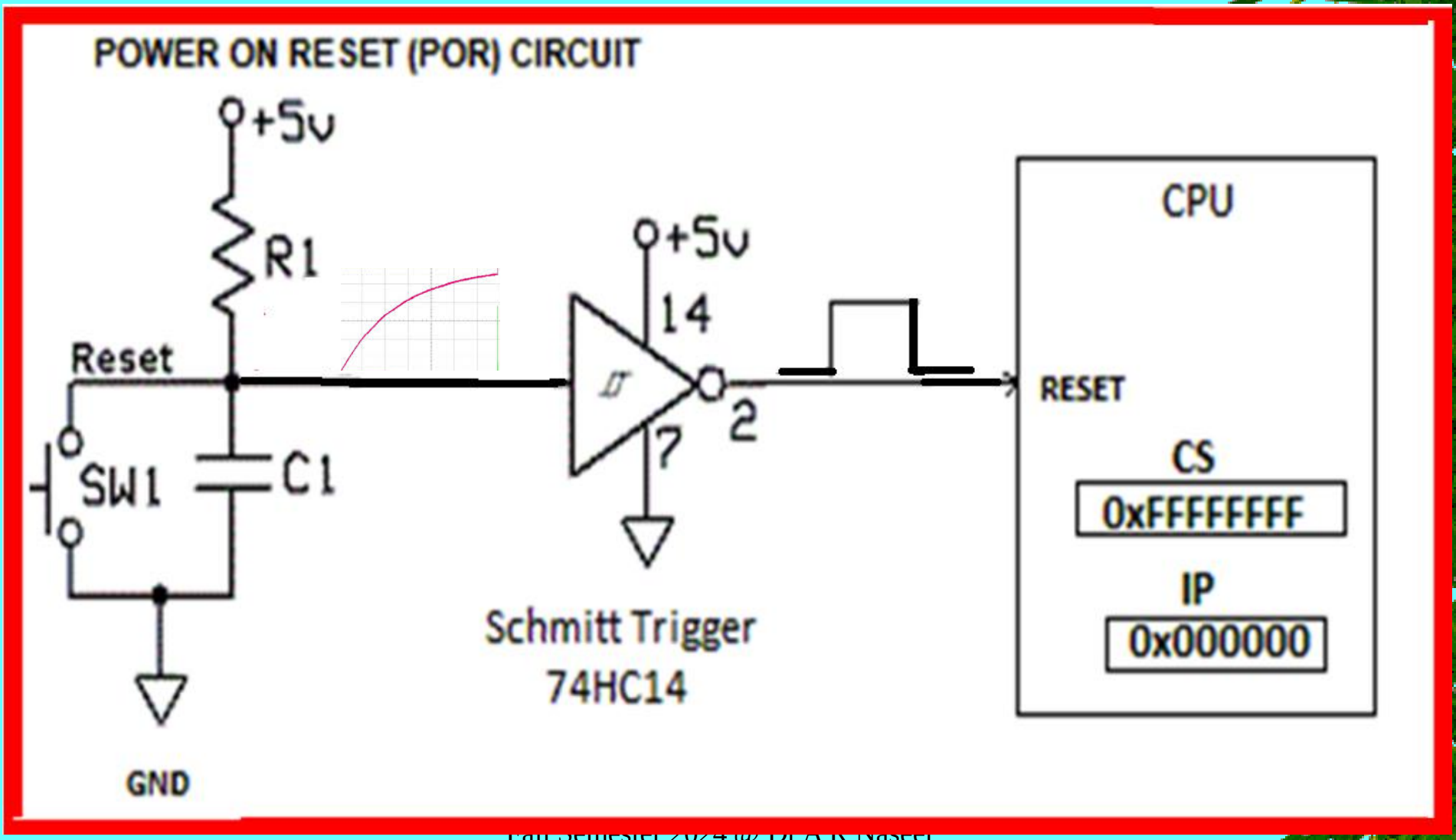
- **Traditionally, the term bootstrap refers to a person who tries to stand up by pulling her own boots. The term bootstrapping dates back to 18th century. Tall boots may have a tab, loop or handle at the top known as a bootstrap, allowing one to use fingers or a boot hook tool to help pulling the boots on.**
- **In operating systems, the term denotes bringing at least a portion of the operating system into main memory and having the processor execute it.**
- **It also denotes the initialization of kernel data structures, the creation of some user processes, and the transfer of control to one of them.**
- **Computer bootstrapping is a tedious, long task, since initially nearly every hardware device including the RAM is in a random, unpredictable state.**
- **Moreover, the bootstrap process is highly dependent on the computer architecture; usually referred to IBM's PC architecture**

SYSTEM STARTUP

BIOS

- The moment after a computer is powered on, it is practically useless because the RAM chips contain random data and no operating system is running.
- To begin the boot, a special hardware circuit known as POWER ON RESET(POR) circuit raises the logical value of the RESET pin of the CPU, i.e. logical value of the RESET pin is raised to Logic ‘1’.
- After RESET is thus asserted, some registers of the processor (including cs and eip) are set to fixed values, i.e., CS=0xFFFFFFFF & IP=0x00000 and the code found at physical address 0xFFFFFFF0 is executed.
- This address is mapped by the hardware to some read-only, persistent memory chip, a kind of memory often called ROM (Read-Only Memory) – EEPROM nowadays holding the firmware.
- The set of programs stored in ROM is traditionally called BIOS (Basic Input/Output System), since it includes several interrupt-driven low-level procedures used by some operating systems, including Microsoft's MS-DOS, to handle the hardware devices that make up the computer.

SYSTEM STARTUP
POWER ON RESET(POR) CIRCUIT



SYSTEM STARTUP

POWER ON RESET (POR) CIRCUIT

- **Power On Reset (POR) ensures that the microprocessor will start in the same condition every time that it is powered up or Reset.**
- The most basic POR circuit comprises a Resistor & Capacitor connected together with values tailored so that when power is first applied, the capacitor takes a predictable and constant time to charge up.
- **A simple POR circuit shown in figure uses the charging of a capacitor in series with a resistor to measure a time period during which the rest of the circuit is held in a reset state.**
- A Schmitt trigger may be used to deassert the reset signal cleanly, once the rising voltage of the RC network passes the threshold voltage of the Schmitt trigger.
- The resistor & Capacitor values should be determined so that the charging of the RC network takes long enough that the supply voltage will have stabilized by the time the threshold is reached.
- The Schmitt trigger circuit is used to convert an analog input signal, i.e., output of a RC circuit (rising charging level of a capacitor) to a digital output signal.
- The circuit is named a trigger because the output retains its value until the input changes sufficiently to trigger a change of the Schmitt remains high until the capacitor gets fully charged, i.e., crosses high threshold of the Schmitt trigger.
- In this way, by pressing the RESET switch on the computer generates[7] a high pulse at the RESET pin of the processor thereby resetting the processor.

SYSTEM STARTUP

PROCESSOR REAL MODE

- **Once The processor is reset, it will initialize all internal registers to zero except Code Segment CS register which is made to contain all 1 bits or CS in initialized to 0xFFFFFFFF and Program Counter or Instruction Pointer IP is initialized to 0, ie., IP=0x00000.**
- **Please note that there are TWO Operating Modes of the processor. They are REAL mode and PROTECTED Mode.**
- **When the processor is reset, it will be operating in REAL mode with minimum features.**
- **In Real mode, the processor generates LOGICAL address which consists of TWO parts – Segment Address & Offset.**
- **The Physical Address of the main memory is computed from logical address by shifting Segment address left by 4 bits or multiplying Segment address by 16 and then adding it to offset.**
- **In order to access the Code segment in memory where you have the program to be executed, the segment address in CS register is multiplied by 16 and then added to Offset in IP register**

INTEL CPU REAL MODE - COMPUTATION of Physical Address of Main Memory :
CS * 16 + IP = 0xFFFFFFFF * 16 + 0x00000 = shift left CS by 4 bits then add IP
= 0xFFFFFFF0 + 0X00000 = 0xFFFFFFF0
JUMP TO LOCATION 0xFFFFFFF0 IN ROM to access BIOS Bootstrap Procedure

SYSTEM STARTUP

BIOS - Basic Input Output System

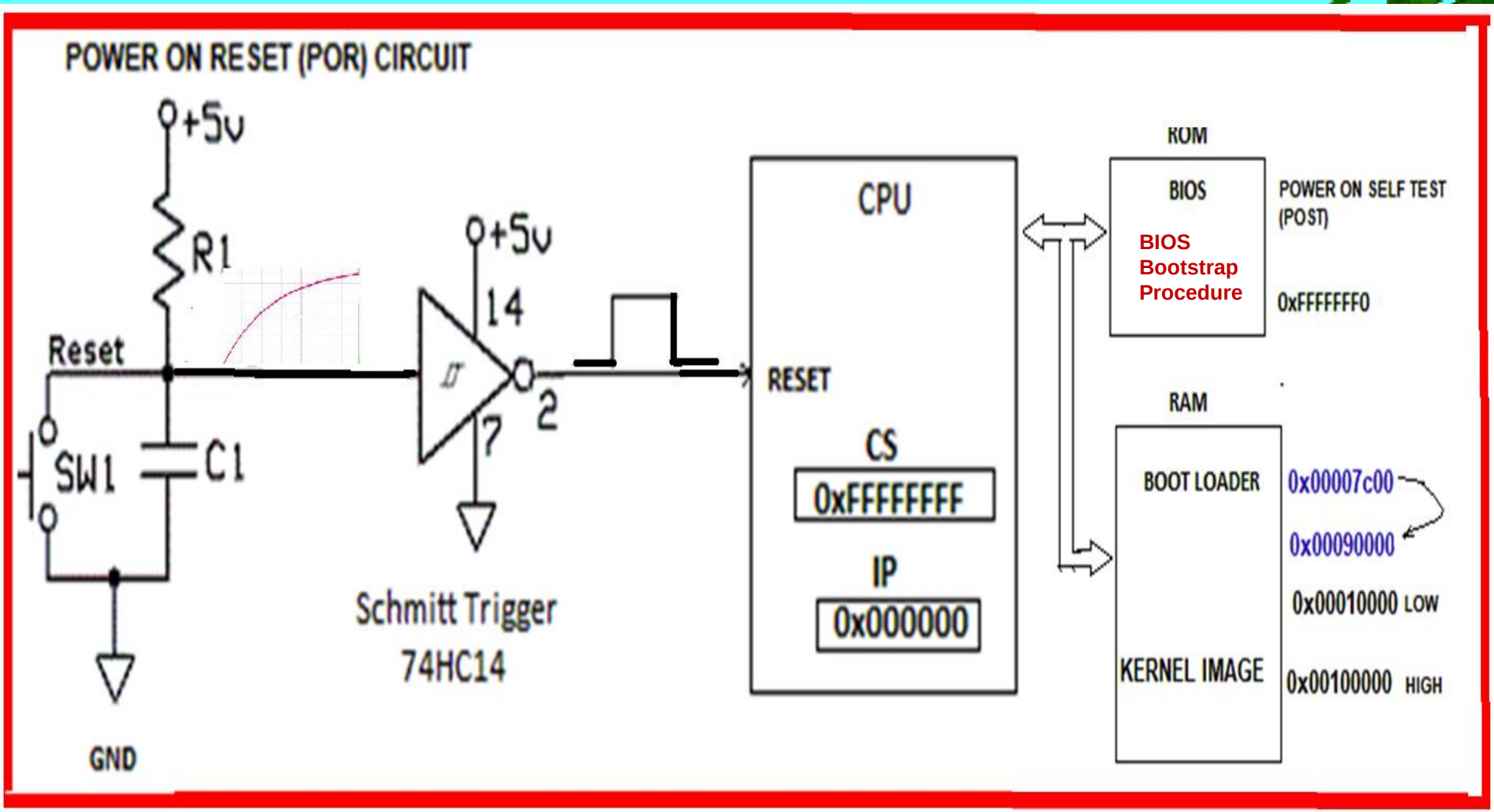
- BIOS is a low-level software (Firmware) that resides in a EEPROM chip on computer's motherboard.
- The BIOS loads when computer starts up, and the BIOS is responsible for waking up computer's hardware components, ensures that they are functioning properly, and then runs the boot loader that boots Windows or Linux, whatever other operating system you have installed.
- The BIOS goes through a POST, or Power-On Self Test, before booting your operating system.
- It checks to ensure your hardware configuration is valid and working properly.
- If something is wrong, you'll see an error message or hear a cryptic series of beep codes.
- When your computer boots—and after the POST finishes—the BIOS looks for a Master Boot Record, or MBR, stored on the boot device and uses it to launch the boot loader.

SYSTEM STARTUP

BIOS(Basic Input Output System)

- Once initialized, Linux does not make any use of BIOS but provides its own device driver for every hardware device on the computer.
- In fact, the BIOS procedures must be executed in real mode, while the kernel executes in protected mode so they cannot share functions even if that would be beneficial.
- BIOS uses Real Mode addresses because they are the only ones available when the computer is turned on.
- A Real Mode address is composed of a seg segment and an off offset; the corresponding physical address is given by seg *16+off.
- As a result, no Global Descriptor Table, Local Descriptor Table, or paging table is needed by the CPU addressing circuit to translate a logical address into a physical one.
- Clearly, the code that initializes the GDT, LDT, and paging tables must run in Real Mode.
- Linux is forced to use BIOS in the bootstrapping phase, when it must retrieve the kernel image from disk or from some other external device.

SYSTEM STARTUP



INTEL CPU REAL MODE - COMPUTATION of Physical Address of Memory :

CS * 16 + IP = 0xFFFFFFFF * 16 + 0x000000 = shift left CS by 4 bits then add IP

= 0xFFFFFFF0 + 0X00000 = 0xFFFFFFF0

JUMP TO LOCATION 0xFFFFFFF0 IN ROM to access BIOS Bootstrap Procedure

SYSTEM STARTUP

- **BIOS Bootstrap Procedure** The BIOS bootstrap procedure essentially performs the following four operations:

1. Executes a series of tests on the computer hardware, in order to establish which devices are present and whether they are working properly. This phase is often called POST (Power-On Self-Test). During this phase, several messages, such as the BIOS version banner, are displayed.

   Recent 80x86, AMD64, and Itanium computers make use of the Advanced Configuration and Power Interface (ACPI) standard. The bootstrap code in an ACPI compliant BIOS builds several tables that describe the hardware devices present in the system. These tables have a vendor-independent format and can be read by the operating system kernel to learn how to handle the devices.

2. Initializes the hardware devices. This phase is crucial in modern PCI-based architectures, since it guarantees that all hardware devices operate without conflicts on the IRQ lines and I/O ports. At the end of this phase, a table of installed PCI devices is displayed.

3. Searches for an operating system to boot. Actually, depending on the BIOS setting, the procedure may try to access (in a predefined, customizable order) the first sector (boot sector) of any floppy disk, any hard disk, and any CD-ROM in the system.

4. As soon as a valid device is found, copies the contents of its first sector, that is, Boot Loader into RAM, starting from physical address 0x00007c00, then jumps into that address and executes the code just loaded.

SYSTEM STARTUP

- **Boot Loader**
  - **The boot loader is the program invoked by the BIOS to load the image of an operating system kernel into RAM.**
- **Working of boot loaders in IBM's PC architecture**
  - **In order to boot from a floppy disk, the instructions stored in its first sector are loaded in RAM and executed; these instructions copy all the remaining sectors containing the kernel image into RAM.**
  - **Booting from a hard disk is done differently. The first sector of the hard disk, named the Master Boot Record (MBR), includes the partition table and a small program, which loads the first sector of the partition containing the operating system to be started. Each partition table entry typically includes the starting and ending sectors of a partition and the kind of operating system that handles it.**
  - **Some operating systems such as Microsoft Windows 98 identify this partition by means of an active flag included in the partition table; following this approach, only the operating system whose kernel image is stored in the active partition can be booted. The active flag may be set through programs like MS-DOS's FDISK**
  - **Linux is more flexible since it replaces the rudimentary program included in the MBR with a sophisticated program called LILO that allows users to select the operating system to be booted.**

SYSTEM STARTUP

- **Booting Linux from Floppy Disk**
- **The only way to store a Linux kernel on a single floppy disk is to compress the kernel image. As we shall see, compression is done at compile time and decompression by the loader.**
- **If the Linux kernel is loaded from a floppy disk, the boot loader is quite simple. It is coded in the arch/i386/boot/bootsect.s assembly language file.**
- **When a new kernel image is produced by compiling the kernel source, the executable code yielded by this assembly language file is placed at the beginning of the kernel image file.**
- **Thus, it is very easy to produce a bootable floppy containing the Linux kernel. The floppy can be created by copying the kernel image starting from the first sector of the disk. When the BIOS loads the first sector of the floppy disk, it actually copies the code of the boot loader.**

SYSTEM STARTUP

- **Booting Linux from Floppy Disk**
  - **The boot loader, which is invoked by the BIOS by jumping to physical address 0x00007c00, performs the following operations:**

1. **Moves itself from address 0x00007c00 to address 0x00090000.**
2. **Sets up the Real Mode stack, from address 0x00003FF4. As usual, the stack will grow toward lower addresses.**
3. **Sets up the disk parameter table, used by the BIOS to handle the floppy device driver.**
4. **Invokes a BIOS procedure to display a "Loading" message.**
5. **Invokes a BIOS procedure to load the setup() code of the kernel image from the floppy disk and puts it in RAM starting from address 0x00090200.**
6. **Invokes a BIOS procedure to load the rest of the kernel image from the floppy disk and puts the image in RAM starting from either low address 0x00010000 (for small kernel images compiled with make zImage) or high address 0x00100000 (for big kernel images compiled with make bzImage).**

SYSTEM STARTUP

- **Booting Linux from Hard Disk**
- **In most cases, the Linux kernel is loaded from a hard disk, and a two-stage boot loader is required.**
- **The most commonly used Linux boot loader on Intel systems is named LILO (LInux LOader)**
- **Other boot loaders for 80 × 86 systems do exist; for instance, the GRand Unified Bootloader (GRUB) is also widely used. GRUB is more advanced than LILO, because it recognizes several disk-based filesystems and is thus capable of reading portions of the boot program from files. Of course, specific boot loader programs exist for all architectures supported by Linux.**
- **LILO may be installed either on the MBR, replacing the small program that loads the boot sector of the active partition, or in the boot sector of a (usually active) disk partition.**
- **In both cases, the final result is the same: when the loader is executed at boot time, the user may choose which operating system to load.**

SYSTEM STARTUP

- **Booting Linux from Hard Disk**

- **Actually, the LILO boot loader is too large to fit into a single sector. Thus the LILO boot loader is broken into two parts, since otherwise it would be too large to fit into the MBR.**
- **The MBR or the partition boot sector includes a small boot loader, which is loaded into RAM starting from address 0x00007c00 by the BIOS.**
- **This small program moves itself to the address 0x00090000, sets up the Real Mode stack (ranging from 0x00098000 to 0x000969ff) and loads the second part of the LILO boot loader into RAM starting from address 0x00090200 and jumps to it.**
- **In turn, this latter program reads a map of available operating systems from disk and offers the user a prompt so he/she can choose one of the operating systems.**
- **Finally, after the user has chosen the kernel to be loaded (or let a time-out elapse so that LILO chooses a default), the boot loader may either copy the boot sector of the corresponding partition into RAM and execute it or directly copy the kernel image into RAM.**

SYSTEM STARTUP

- **Booting Linux from Hard Disk**

Assuming that a Linux kernel image must be booted, the LILO boot loader, which relies on BIOS routines, performs essentially the following operations:

1. **Invokes a BIOS procedure to display a "Loading" message.**
2. **Invokes a BIOS procedure to copy the integrated boot loader of the kernel image (512 bytes or 1 sector size) to address 0x00090000, while the code of the setup( ) function is put in RAM starting from address 0x00090200.**
3. **Invokes a BIOS procedure to load the rest of the kernel image from disk and puts the image in RAM starting from either low address 0x00010000 (for small kernel images compiled with make zImage) or high address 0x00100000 (for big kernel images compiled with make bzImage).**
4. **Jumps to the setup( ) code.**

- **The code of the setup() assembly language function is placed by the linker immediately after the integrated boot loader of the kernel, that is, at offset 0x200 of the kernel image file. The boot loader can thus easily locate the code and copy it into RAM starting from physical address 0x00090200.**

SYSTEM STARTUP

setup() function

- The code of the setup( ) function is put in RAM starting from address 0x00090200 and it is now being executed.
- The setup( ) function must initialize the hardware devices in the computer and set up the environment for the execution of the kernel program.
- Although the BIOS already initialized most hardware devices, Linux does not rely on it but reinitializes the devices in its own manner to enhance portability and robustness.

SYSTEM STARTUP

- **setup() function**
- **setup( ) performs essentially the following operations:**

1. **In ACPI-compliant systems, it invokes a BIOS routine that builds a table in RAM describing the layout of the system's physical memory. In older systems, it invokes a BIOS routine that just returns the amount of RAM available in the system.**
2. **Sets the keyboard repeat delay and rate. (When the user keeps a key pressed past a certain amount of time, the keyboard device sends the corresponding keycode over and over to the CPU.)**

   Keyboard default rate is 10.9cps (characters per second) & repeat delay is 250ms. Linux command to change the rate & repeat delay is

   $kbdrate –r rate –d delay

   The repeat delay is the amount of time that a key must be depressed before it will start to repeat.
3. **Initializes the video adapter card.**
4. **Reinitializes the disk controller and determines the hard disk parameters.**
5. **Checks for an IBM Micro Channel bus (MCA).**

SYSTEM STARTUP

- **SETUP function** essentially performs the following operations:

6. Checks for a PS/2 pointing device (bus mouse).
7. Checks for Advanced Power Management (APM) BIOS support.
8. If the BIOS supports the *Enhanced Disk Drive Services* (*EDD*), it invokes the proper BIOS procedure to build a table in RAM describing the hard disks available in the system. (The information included in the table can be seen by reading the files in the *firmware/edd* directory of the *sysfs* special filesystem.)
9. If the kernel image was loaded low in RAM (at physical address 0x0001000), the function moves it to physical address 0x0000100000. Conversely, if the kernel image was loaded high in RAM, the function does not move it. This step is necessary because to be able to store the kernel image on a floppy disk and to reduce the booting time, the kernel image stored on disk is compressed, and the decompression routine needs some free space to use as a temporary buffer following the kernel image in RAM.
10. Sets the A20 pin located on the 8042 keyboard controller. The A20 pin is a hack introduced in the 80286-based systems to make physical addresses compatible with those of the ancient 8088 microprocessors. Unfortunately, the A20 pin must be properly set before switching to Protected Mode, otherwise the 21st bit of every physical address will always be regarded as zero by the CPU.
11. Sets up a provisional Interrupt Descriptor Table (IDT) and a provisional Global Descriptor Table (GDT).
12. Resets the floating point unit (FPU), if any.
13. Reprograms the Programmable Interrupt Controller (PIC) and maps the 16 hardware interrupts (IRQ lines) to the range of vectors from 32 to 47. The kernel must perform this step because the BIOS erroneously maps the hardware interrupts in the range from 0 to 15, which is already used for CPU exceptions. It also masks all interrupts except IRQ2 which is the cascading interrupt between the two PICs.
14. Switches the CPU from Real Mode to Protected Mode by setting the PE bit in the cr0 status register. The provisional kernel page tables contained in swapper_pg_dir and pg0 identically map the linear addresses to the same physical addresses. Therefore, the transition from Real Mode to Protected Mode goes smoothly. The PG bit in the cr0 register is cleared, so paging is still disabled.
15. Jumps to the startup_32( ) assembly language function.

SYSTEM STARTUP

FIRST Startup function

There are two different startup_32( ) functions; the one we refer to here is coded in the arch/i386/boot/compressed/head.S file.

After setup() terminates, the function has been moved either to physical address 0x00100000 or to physical address 0x00001000, depending on whether the kernel image was loaded high or low in RAM.

The first startup_32 function performs the following operations:

1. Initializes the segmentation registers and a provisional stack.
2. Clears all bits in the eflags register
3. Fills the area of uninitialized data of the kernel identified by the _edata and _end symbols with zeros.
4. Invokes the decompress_kernel() function to decompress the kernel image. The "Uncompressing Linux . . . " message is displayed first. After the kernel image has been decompressed, the "O K, booting the kernel." message is shown. If the kernel image was loaded low, the decompressed kernel is placed at physical address 0x00100000. Otherwise, if the kernel image was loaded high, the decompressed kernel is placed in a temporary buffer located after the compressed image. The decompressed image is then moved into its final position, which starts at physical address 0x00100000.
5. Jumps to physical address 0x00100000. The decompressed kernel image begins with another startup_32( ) function included in the arch/i386/kernel/head.S file. Using the same name for both the functions does not create any problems, since both functions are executed by jumping to their initial physical addresses.

SYSTEM STARTUP

- **SECOND Startup function**

The second startup_32() function essentially sets up the execution environment for the first Linux process (process 0).

The function performs the following operations:

1. **Initializes the segmentation registers with their final values.**
2. **Fills the bss segment of the kernel with zeros.**
3. **Initializes the provisional kernel Page Tables contained in swapper_pg_dir and pg0 to identically map the linear addresses to the same physical addresses,**
4. **Stores the address of the Page Global Directory in the cr3 register, and enables paging by setting the PG bit in the cr0 register.**
5. **Sets up the Kernel Mode stack for process (refer Chapter 3).**
6. **Once again, the function clears all bits in the eflags register.**
7. **Invokes setup_idt() to fill the IDT with null interrupt handlers (refer Chapter4).**
8. **Puts the system parameters obtained from the BIOS and the parameters passed to the operating system into the first page frame (refer Chapter 2).**
9. **Identifies the model of the processor.**
10. **Loads the gdtr and idtr registers with the addresses of the GDT and IDT tables.**
11. **Jumps to the start_kernel( ) function.**

SYSTEM STARTUP

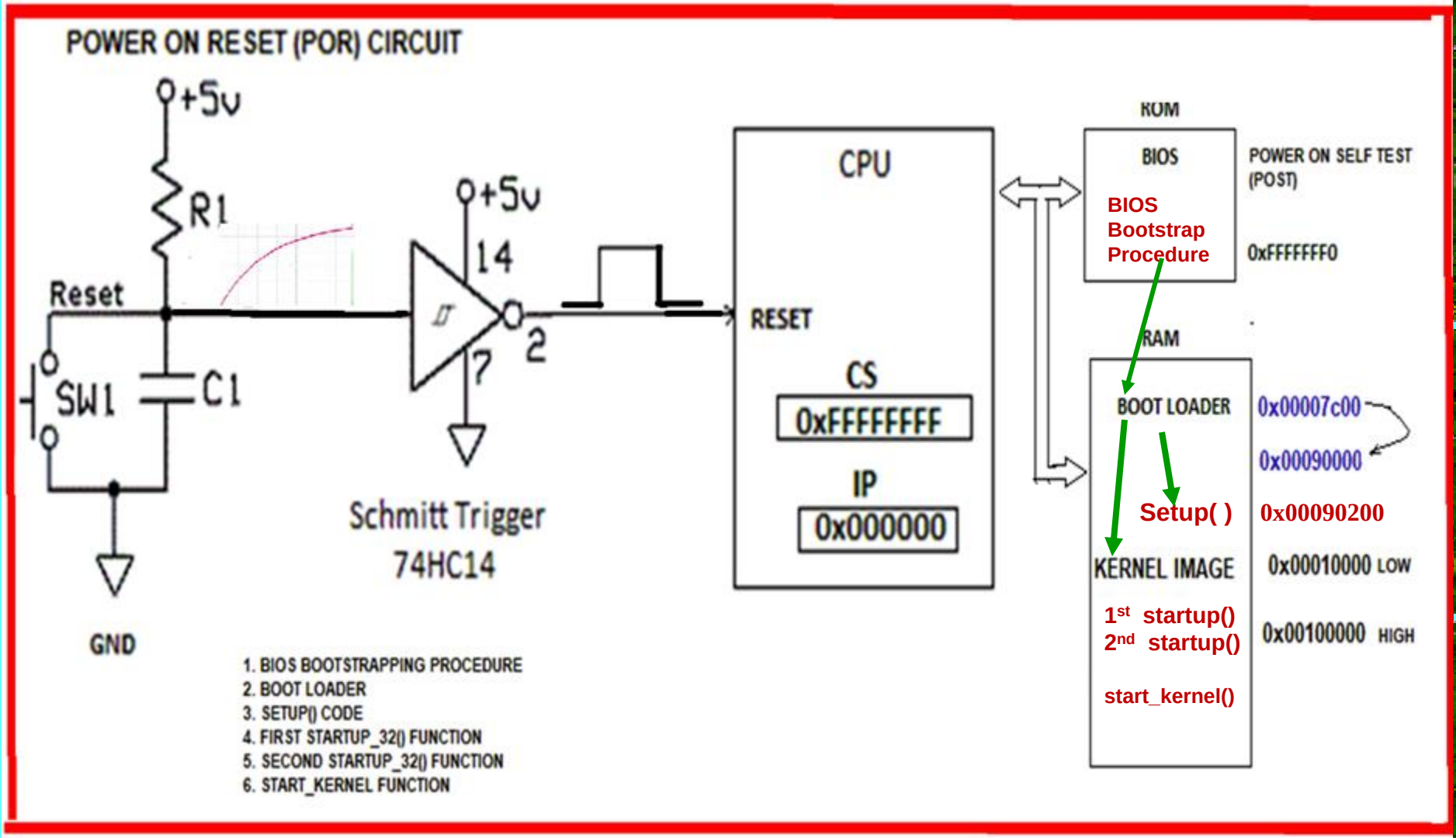
Start-kernel() function

start_kernel( ) function completes the initialization of the Linux kernel. Nearly every kernel component is initialized by this function. It creates the PROCESS 1 (init)

- The scheduler is initialized by invoking the sched_init() function (seeChapter 7).
- The memory zones are initialized by invoking the build_all_zonelists() function (see Chapter 8).
- The Buddy system allocators are initialized by invoking the page_alloc_init() and mem_init() functions (see Chapter 8).
- **The final initialization of the IDT is performed by invoking trap_init() and init_IRQ() (refer Chapter 4).**
- **The TASKLET_SOFTIRQ and HI_SOFTIRQ are initialized by invoking the softirq_init() function (see the section "Softirqs" in Chapter 4).**
- **The page tables are initialized by invoking the paging_init( ) function (refer Chapter 2).**
- **The page descriptors are initialized by the mem_init( ) function (refer Chapter 6).**
- **The slab allocator is initialized by the kmem_cache_init() and kmem_cache_sizes_init() functions (refer Chapter 6).**
- **•The system date and time are initialized by the time_init() function (refer Chapter 5).**
- The speed of the CPU clock is determined by invoking the calibrate_delay() function (see Chapter 6)
- **The kernel thread for process 1 is created by invoking the kernel_thread( ) function. In turn, this kernel thread creates the other kernel threads and executes the /sbin/init program, as described in Chapter 3.**

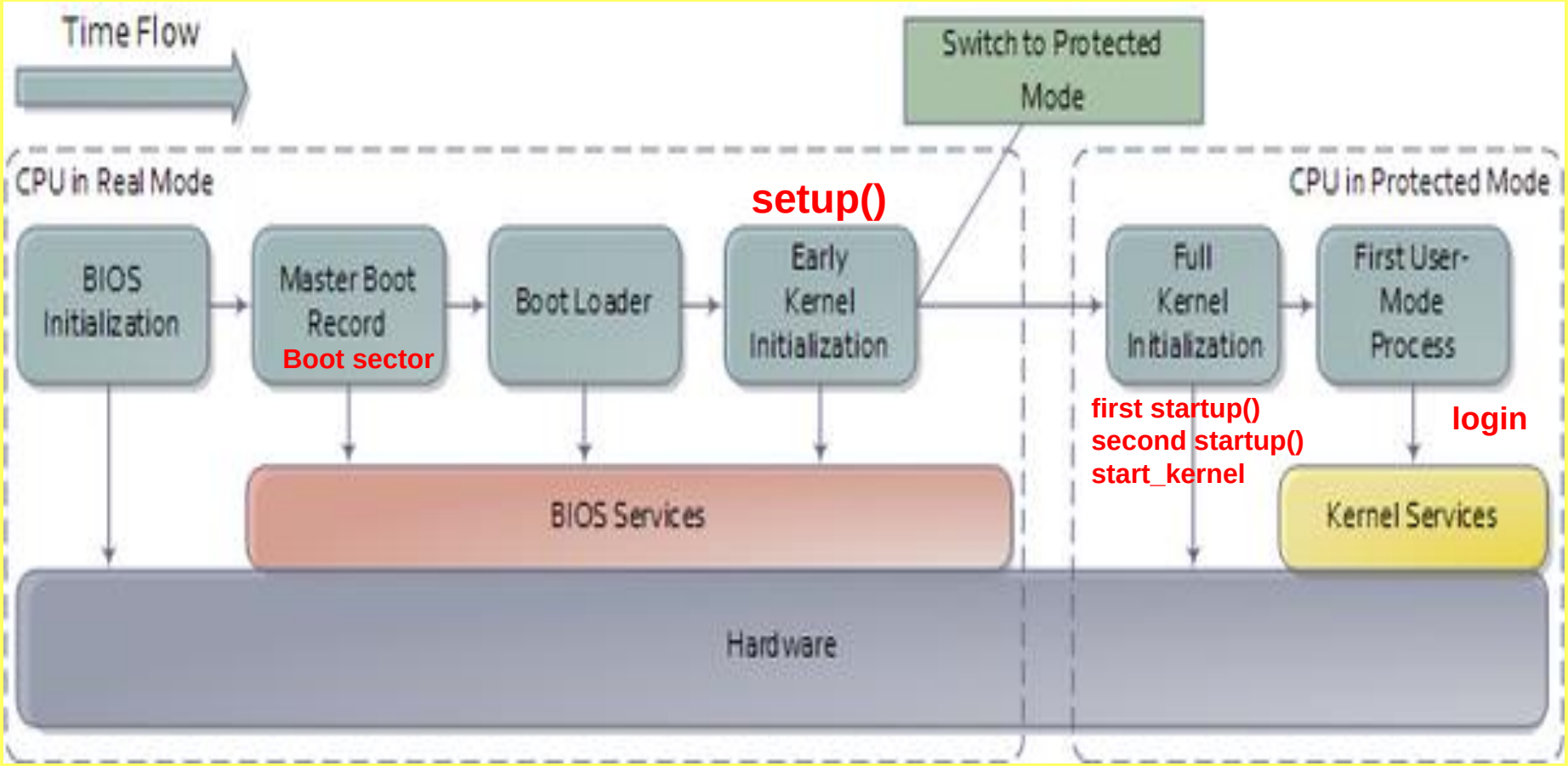
Besides the "Linux version 4.xx.yy . . . " message, which is displayed right after the beginning of start_kernel( ), many other messages are displayed in this last phase both by the init functions and by the kernel threads. At the end, the familiar login prompt appears on the console (or in the graphical screen if the X Window System is launched at startup), telling the user that the Linux kernel is up and running.

SYSTEM STARTUP



SYSTEM STARTUP

BOOTING PROCESS IN LINUX



SYSTEM STARTUP

BIOS IS OUTDATED

- The BIOS has been around for a long time, and has not evolved much.
- Even MS-DOS PCs released in the 1980s had a BIOS. Of course, the BIOS has evolved and improved over time.
- Some extensions were developed, including ACPI, the Advanced Configuration and Power Interface. This allows the BIOS to more easily configure devices and perform advanced power management functions, like sleep.
- But the BIOS has not advanced and improved nearly as much as other PC technology has since the days of MS-DOS.
- The traditional BIOS still has serious limitations.
- It can only boot from drives of 2.1 TB or less.
- 3 TB drives are now common, and a computer with a BIOS can't boot from them.
- That limitation is due to the way the BIOS's Master Boot Record system works.
- The BIOS must run in 16-bit processor mode, and only has 1 MB of space to execute in.
- It has trouble initializing multiple hardware devices at once, which leads to a slower boot process when initializing all the hardware interfaces and devices on a modern PC.

SYSTEM STARTUP

UEFI REPLACES BIOS WITH IMPROVEMENTS

- In 2007, Intel, AMD, Microsoft, and PC manufacturers agreed on a new Unified Extensible Firmware Interface (UEFI) specification.
- This is an industry-wide standard managed by the Unified Extended Firmware Interface Forum, and is not solely driven by Intel.
- UEFI support was introduced to Windows with Windows Vista Service Pack 1 and Windows 7.
- The vast majority of computers you can buy today now use UEFI rather than a traditional BIOS.
- UEFI replaces the traditional BIOS on PCs. There is no way to switch from BIOS to UEFI on an existing PC.
- Most UEFI implementations provide BIOS emulation so you can choose to install and boot old operating systems that expect a BIOS instead of UEFI, so they are backward compatible.

SYSTEM STARTUP

UEFI REPLACES BIOS WITH IMPROVEMENTS

- **UEFI - new standard that avoids the limitations of the BIOS.**
- **The UEFI firmware can boot from drives of 2.2 TB or larger—in fact, the theoretical limit is 9.4 zetta bytes. That's roughly three times the estimated size of all the data on the Internet. That's because UEFI uses the GPT partitioning scheme instead of MBR.**
- **It also boots in a more standardized way, launching EFI executables rather than running code from a drive's master boot record.**
- **UEFI can run in 32-bit or 64-bit mode and has more addressable address space than BIOS, which means your boot process is faster.**
- **It also means that UEFI setup screens can be slicker than BIOS settings screens, including graphics and mouse cursor support.**
- **UEFI is packed with other features. It supports Secure Boot, which means the operating system can be checked for validity to ensure no malware has tampered with the boot process.**
- **It can support networking features right in the UEFI firmware itself, which can aid in remote troubleshooting and configuration.**

SYSTEM STARTUP

MBR (Master Boot Record)

- **MBR (Master Boot Record) is a way of storing the partitioning information on a drive. Partitioning information includes where partitions start and begin, so your operating system knows which sectors belong to each partition and which partition is bootable**

- **MBR contains a very important bit of code known as the "bootstrap code. The first 440 of these 512 bytes contain the bootloader.**
- **Next MBR contains something called the partition table, which is an index of up to four partitions that exist on the same disk and with these we can have different filesystems on the same drive stored in different partitions.**
- **On IBM-compatible PCs, the final two bytes of the 512-byte MBR are called the *boot signature* and are used by the BIOS to determine if the selected boot drive is actually bootable or not. On a disk that contains valid bootstrap code, the last two bytes of the MBR should always be 0x55 0xAA.If the last two bytes of the MBR do not equal 0x55 and 0xAA respectively, the BIOS will assume that the disk is *not* bootable and is not a valid boot option**

SYSTEM STARTUP

- **MBR (Master Boot Record) and GPT (GUID Partition Table)**

- **MBR (Master Boot Record) and GPT (GUID Partition Table) are two different ways of storing the partitioning information on a drive. GUID stands for Globally Unique IDentifier**
- **Partitioning information includes where partitions start and begin, so your operating system knows which sectors belong to each partition and which partition is bootable**
- **A partition structure defines how information is structured on the partition, where partitions begin and end, and also the code that is used during startup if a partition is bootable.**
- **GPT brings with it many advantages, but MBR is still the most compatible and is still necessary in some cases.**
- **This is not a Windows-only standard, by the way—Mac OS X, Linux, and other operating systems also support GPT.**

SYSTEM STARTUP

LIMITATIONS OF MBR

- **MBR was first introduced with IBM PC DOS 2.0 in 1983.**
- **It is called Master Boot Record because the MBR is a special boot sector located at the beginning of a drive.**
- This sector contains a boot loader for the installed operating system and information about the drive's logical partitions.
- The boot loader is a small bit of code that generally loads the larger boot loader from another partition on a drive.
- If you have Windows installed, the initial bits of the Windows boot loader reside here.
- If you have Linux installed, the LILO/GRUB boot loader will typically be located in the MBR.
- **MBR does have its limitations.**
- **MBR only works with disks up to 2 TB in size.**
- **MBR also only supports up to four primary partitions—if you want more, you have to make one of your primary partitions an "extended partition" and create logical partitions inside it.**

SYSTEM STARTUP

- **ADVANTAGES OF GPT**

- **GPT stands for GUID Partition Table.**
- **It is a new standard that is gradually replacing MBR. It is associated with UEFI, which replaces the clunky old BIOS with something more modern.**
- **GPT, in turn, replaces the clunky old MBR partitioning system with something more modern.**
- **It is called GUID Partition Table because every partition on your drive has a "globally unique identifier," or GUID—a random string so long that every GPT partition on earth likely has its own unique identifier.**
- **GPT doesn't suffer from MBR's limits.**
- **GPT-based drives can be much larger, with size limits dependent on the operating system and its file systems.**
- **GPT also allows for a nearly unlimited number of partitions. Again, the limit here will be your operating system—Windows allows up to 128 partitions on a GPT drive, and you don't have to create an extended partition to make them work.**

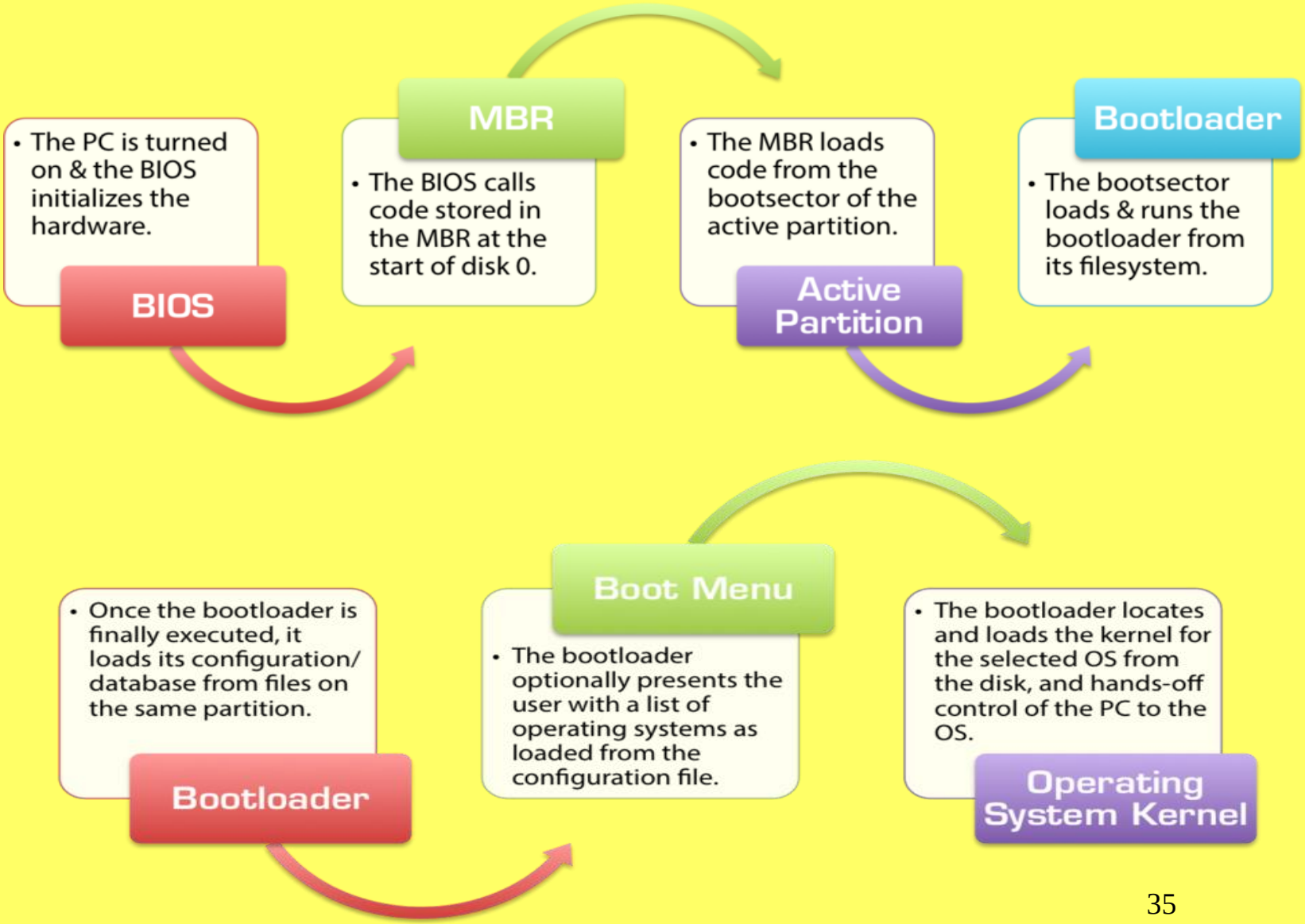
SYSTEM STARTUP

ADVANTAGES OF GPT

- **On an MBR disk, the partitioning and boot data is stored in one place. If this data is overwritten or corrupted, you're in trouble.**
- **In contrast, GPT stores multiple copies of this data across the disk, so it's much more robust and can recover if the data is corrupted.**
- **Windows can only boot from GPT on UEFI-based computers running 64-bit versions of Windows 10, 8, 7, Vista, and corresponding server versions.**
- All versions of Windows 10, 8, 7, and Vista can read GPT drives and use them for data—they just can't boot from them without UEFI.
- **Other modern operating systems support GPT.**
- **Linux has built-in support for GPT.**
- **Apple's Intel Macs no longer use Apple's APT (Apple Partition Table) scheme and use GPT instead.**

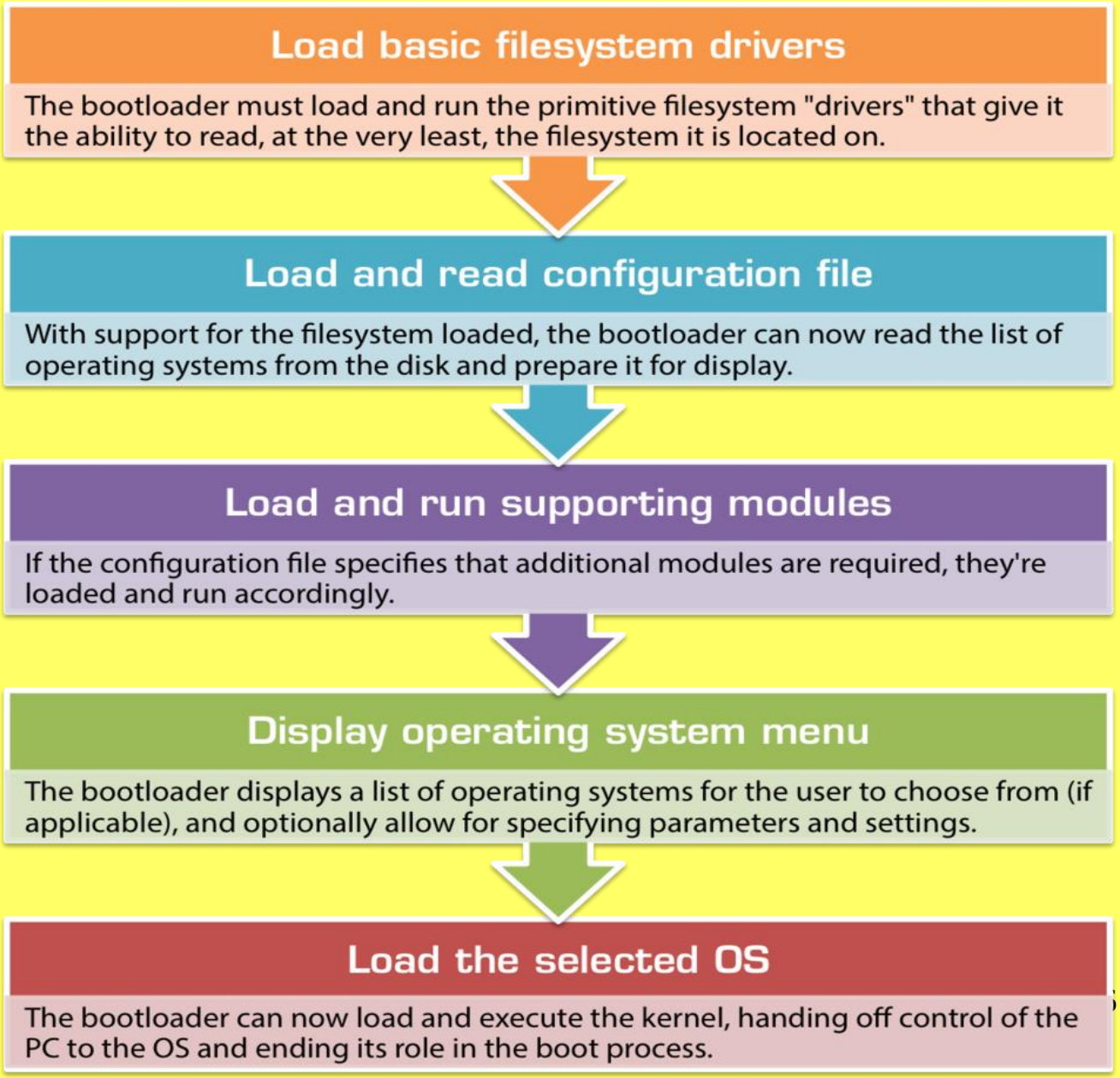
SYSTEM STARTUP

BOOTING PROCESS IN WINDOWS



SYSTEM STARTUP

BOOTLOADER OPERATIONS



SYSTEM STARTUP

BOOT LOADER

NTLDR	BOOTMGR	GRUB(2)
• NTLDR is the default bootloader for Windows NT, 2000, and XP. • BOOT.INI on the active partition contains the list of operating systems and their locations. • NTDETECT.COM is a helper program that runs to detect hardware and identify devices.	• BOOTMGR is the new Windows and is used on Windows Vista, 7, 8, and 10. • The list of operating systems is now read from the BCD file in the BOOT directory on the active partition. • BOOTMGR is self-contained, and does not need any helper programs or routines.	• GRUB is the most-popular bootloader for Linux, though it can boot numerous other OSes as well. • Its boot settings are stored in a file usually called grub.cfg (GRUB2) or menu.lst (GRUB). • GRUB is a modular bootloader, that can load additional modules from disk.

- Each of the popular operating systems has its own default bootloader.
- Windows NT, 2000, and XP as well as Windows Server 2000 and Windows Server 2003 use the **NTLDR bootloader.**
- Windows Vista introduced the **BOOTMGR bootloader**, currently used by Windows Vista, 7, 8, and 10, as well as Windows Server 2008 and 2012.
- While a number of different bootloaders have existed for Linux over the years, the two predominant bootloaders were **LILO and GRUB**, but now most Linux distributions have coalesced around the all-powerful **GRUB2 bootloader**.

SYSTEM STARTUP

NTLDR, BOOTMGR, GRUB, GRUB2

- **NTLDR is the old Windows bootloader**, first used in Windows NT (hence the “NT” in “NTLDR,” short for “NT Loader”), and currently used in Windows NT, Windows 2000, Windows XP, and Windows Server 2003.NTLDR stores its boot configuration in a simple, text-based file called BOOT.INI, stored in the root directory of the active partition (often C:\Boot.ini). Once NTLDR is loaded and executed by the second-stage bootloader, it executes a helper program called NTDETECT.COM that identifies hardware and generates an index of information about the system.
- **BOOTMGR is the newer version of the bootloader used by Microsoft Windows**, and it was first introduced in the beta versions of Windows Vista (then Windows Codename Longhorn). It’s currently used in Windows Vista, Windows 7, Windows 8, Windows 8.1, and Windows 10, as well as Windows Server 2008 and Windows Server 2012. BOOTMGR marked a significant departure from NTLDR. It is a self-contained bootloader with many more options, especially designed to be compatible with newer functionality in modern operating systems and designed with EFI and GPT in mind. Unlike NTLDR, BOOTMGR stores its configuration in a file called the BCD – short for Boot Configuration Database. Unlike BOOT.INI, the BCD file is a binary database that cannot be opened and edited by hand. Instead, specifically designed command-line tools like bcdedit.exe and more user-friendly GUI utilities such as EasyBCD must be used to read and modify the list of operating systems.

SYSTEM STARTUP

NTLDR, BOOTMGR, GRUB, GRUB2

- **GRUB** was the predominantly-used bootloader for Linux in the 1990s and early 2000s, designed to load not just Linux, but any operating system implementing the open multiboot specification for its kernel. GRUB's configuration file containing a whitespace-formatted list of operating systems was often called menu.lst or grub.lst, and found under the /boot/ or /boot/grub/ directory. As these values could be changed by recompiling GRUB with different options, different Linux distributions had this file located under different names in different directories.
- **GRUB 2** -While GRUB eventually won out over Lilo and eLilo, it was replaced with GRUB 2 around 2002, and the old GRUB was officially renamed "Legacy GRUB". GRUB 2 is now officially called GRUB, while the old GRUB has officially been relegated to the name of "Legacy GRUB". GRUB 2 is a powerful, modular bootloader more akin to an operating system than a bootloader. It can load dozens of different operating systems, and supports custom plugins ("modules") to introduce more functionality and support complex boot procedures. The actual bootloader file for GRUB 2 is not a file called GRUB2, but rather a file usually called *core.img*. Unlike Legacy GRUB, the GRUB 2 configuration file is more of a script and less of traditional configuration file. The grub.cfg file, normally located at /boot/grub/grub.cfg on the boot partition, bears resemblance to shell scripts and supports advanced concepts like functions. The core functionality of GRUB 2 is supplemented with modules, normally found in a subdirectory of the /boot/grub/ directory.

SOC 3010 OPERATING SYSTEMS

Reading Assignments

APPENDIX A OF TEXT BOOK

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati**
    Edition, O' Reilly Media 2005, ISBN 978-0596005658

SYSTEM STARTUP

- **Questions**
  - **What is meant by Bootstrapping?**
  - **Describe clearly all the steps in System Startup.**
  - **Write the two operating modes of the Intel processor.**
  - **Explain logical address generated by processor?**
  - **Describe the Physical address computation in Real Mode of Intel CPU.**
  - **Explain clearly all the steps in the BIOS Bootstrap Procedure.**
  - **Describe the operations performed by the Boot Loader.**
  - **Which routine loads the Boot Loader into memory.**
  - **Which routine copies the Kernel image into memory.**
  - **Name the boot loader used by Linux.**

SYSTEM STARTUP

Questions

- Name the boot loader used by WINDOWS NT, 2000, XP
- Name the boot loader used by WINDOWS VISTA, 7,8, 10
- Explain GRUB & GRUB2?
- Write the full form of the following abbreviations :

a) LILO	b) MBR	c) GPT	d)GUID
e) UUID	f) GRUB	g) UEFI	h) BIOS
i) POST	j) NTLDR	k)BOOTMGR	

- Describe the operations performed by setup() function.
- Which function switches the CPU from Real Mode to Protected Mode by setting the PE bit in the cr0 control register.
- Setup() function switches the CPU from Real Mode to Protected Mode by setting the ……bit in the …….. control register.
- Which function decompresses the kernel image and loads it into memory starting at physical address 0x00100000?

SYSTEM STARTUP

Questions

- What are the operations performed by first startup() function?
- What are the operations performed by second startup() function?
- Which function sets up the execution environment for the Linux process 0?
- Which function Initializes the provisional kernel Page Tables contained in swapper_pg_dir and pg0 to identically map the linear addresses to the same physical addresses?
- Which process loads the gdtr and idtr registers with the addresses of the GDT and IDT tables
- Which process is created by start_kernel() function?
- What are the operations performed by start_kernel() function?

SOC 3010
OPERATING SYSTEM

END OF

WEEK 2 LECTURE 1

SYSTEM STARTUP

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG